

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №2
“Объектно – ориентированные возможности языка Python”

Выполнил:
Студент группы ИУ5-33Б
Якубова Д. Р.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Цель лабораторной работы

Цель лабораторной работы: изучение объектно-ориентированных возможностей языка Python.

Задание лабораторной работы

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием `pip`.
2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке Python 3.
3. Все файлы проекта (кроме основного файла `main.py`) должны располагаться в пакете `lab_python_oop`.
4. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета `lab_python_oop`.
5. Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры.
6. Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры.
7. Класс «Прямоугольник» наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры.
8. Класс «Круг» создается аналогично классу «Прямоугольник», задается параметр «радиус». Для вычисления площади используется константа `math.pi` из модуля `math`.
9. Класс «Квадрат» наследуется от класса «Прямоугольник». Класс должен содержать конструктор по длине стороны. Для классов «Прямоугольник», «Квадрат», «Круг»:
 - Определите метод `"repr"`, который возвращает в виде строки основные параметры фигуры, ее цвет и площадь. Используйте метод `format`.
 - Название фигуры («Прямоугольник», «Квадрат», «Круг») должно задаваться в виде поля данных класса и возвращаться методом класса.

10. В корневом каталоге проекта создайте файл `main.py` для тестирования Ваших классов. Создайте следующие объекты и выведите о них информацию в консоль (N - номер Вашего варианта по списку группы):

- Прямоугольник синего цвета шириной N и высотой N.
- Круг зеленого цвета радиусом N.
- Квадрат красного цвета со стороной N.
- Также вызовите один из методов внешнего пакета, установленного с использованием `pip`.

11. **Дополнительное задание.** Протестируйте корректность работы Вашей программы с помощью модульного теста.

Листинг кода (доп задание):

`Circle.py`

```
from lab_python_oop.figure import Figure
from lab_python_oop.color import FigureColor
import math

class Circle(Figure):

    FIGURE_TYPE = "Круг"

    def __init__(self, color_parameter: str, radius_parameter: float):
        if radius_parameter <= 0:
            raise ValueError("Радиус должен быть положительным числом")
        self._radius = radius_parameter
        self._figure_color = FigureColor(color_parameter)

    @classmethod
    def get_name(cls) -> str:
        return cls.FIGURE_TYPE

    def get_color(self) -> str:
        return self._figure_color.get_color()

    def get_radius(self) -> float:
        return self._radius

    def square(self) -> float:
        return math.pi*(self._radius**2)

    def __repr__(self):
        return '{} {} цвета радиусом {} площадью {}'.format(
            Circle.get_name(),
```

```

        self._figure_color.get_color(),
        self._radius,
        self.square()
    )

```

Color.py

```

class FigureColor:

    def __init__(self, color: str = None):
        self._color = color

    def get_color(self) -> str:
        return self._color

    def set_color(self, value: str):
        if not value or not isinstance(value, str):
            raise ValueError("Поле цвета не может быть пустым")
        self._color = value

```

Figure.py

```

from abc import ABC, abstractmethod

```

```

class Figure(ABC):

    @abstractmethod
    def square(self) -> float:
        pass

    @classmethod
    @abstractmethod
    def get_name(cls) -> str:
        pass

    @abstractmethod
    def get_color(self) -> str:
        pass

    @abstractmethod
    def __repr__(self) -> str:
        pass

```

Rectangle.py

```

from lab_python_oop.figure import Figure
from lab_python_oop.color import FigureColor

```

```

class Rectangle(Figure):

```

```

FIGURE_TYPE = "Прямоугольник"

def __init__(self, color_parameter: str, width_parameter: float,
height_parameter: float):
    if width_parameter <= 0 or height_parameter <=0:
        raise ValueError("Ширина и высота должны быть положительными
числами")
    self._width = width_parameter
    self._height = height_parameter
    self._figure_color = FigureColor(color_parameter)

    @classmethod
    def get_name(cls):
        return cls.FIGURE_TYPE

    def get_color(self):
        return self._figure_color.get_color()

    def get_width(self):
        return self._width

    def get_height(self):
        return self._height

    def square(self):
        return self._width*self._height

    def __repr__(self):
        return '{} {} цвета шириной {} и высотой {} площадью {}'.format(
            Rectangle.get_name(),
            self._figure_color.get_color(),
            self._width,
            self._height,
            self.square()
        )

```

Square.py

```

from lab_python_oop.rectangle import Rectangle

```

```

class Square(Rectangle):

```

```

    FIGURE_TYPE = "Квадрат"

```

```

    def __init__(self, color_parameter, side_parameter):
        if side_parameter <= 0:
            raise ValueError("Сторона должна быть положительным числом")
        self._side = side_parameter
        super().__init__(color_parameter, self._side, self._side)

```

```

@classmethod
def get_name(cls):
    return cls.FIGURE_TYPE

def get_side(self) -> float:
    return self._side

def __repr__(self):
    return '{} {} цвета со стороной {} площадью {}'.format(
        Square.get_name(),
        self._figure_color.get_color(),
        self._side,
        self.square()
    )

```

Main.py

```

from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

def main():
    rectangle = Rectangle("синего", 1, 1)
    circle = Circle("зеленого", 1)
    square = Square("красного", 1)
    print(rectangle)
    print(circle)
    print(square)

if __name__ == "__main__":
    main()

```

Test.py

```

import pytest
import math
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square
from lab_python_oop.color import FigureColor

class TestFigures:
    @pytest.fixture
    def sample_rectangle(self):
        return Rectangle("синего", 5, 3)

    @pytest.fixture
    def sample_circle(self):
        return Circle("зеленого", 4)

```

```

@pytest.fixture
def sample_square(self):
    return Square("красного", 2)

def test_rectangle_creation_and_square(self, sample_rectangle):
    assert sample_rectangle.square() == 15
    assert sample_rectangle.get_color() == "синего"
    assert sample_rectangle.get_width() == 5
    assert sample_rectangle.get_height() == 3

def test_circle_creation_and_square(self, sample_circle):
    assert sample_circle.square() == pytest.approx(math.pi * 16, 0.01)
    assert sample_circle.get_color() == "зеленого"
    assert sample_circle.get_radius() == 4

def test_rectangle_creation_and_square_and_isinstance(self, sample_square):
    assert sample_square.square() == 4
    assert sample_square.get_color() == "красного"
    assert sample_square.get_side() == 2
    assert isinstance(sample_square, Rectangle)

@pytest.mark.parametrize("width,height,expected_error", [
    (-1, 5, "Ширина и высота должны быть положительными числами"),
    (0, 5, "Ширина и высота должны быть положительными числами"),
    (5, -1, "Ширина и высота должны быть положительными числами"),
    (-5, -1, "Ширина и высота должны быть положительными числами"),
])

def test_rectangle_invalid_parameters(self, width, height, expected_error):
    with pytest.raises(ValueError, match=expected_error):
        Rectangle("синего", width, height)

@pytest.mark.parametrize("radius,expected_error", [
    (-1, "Радиус должен быть положительным числом"),
    (0, "Радиус должен быть положительным числом"),
])

def test_circle_invalid_parameters(self, radius, expected_error):
    with pytest.raises(ValueError, match=expected_error):
        Circle("зеленого", radius)

@pytest.mark.parametrize("side,expected_error", [
    (-1, "Сторона должна быть положительным числом"),
    (0, "Сторона должна быть положительным числом"),
])

def test_square_invalid_parameters(self, side, expected_error):
    with pytest.raises(ValueError, match=expected_error):
        Square("красного", side)

```

```

class TestFigureColorTDD:

```

```

def test_color_operations(self):
    color = FigureColor("синий")
    assert color.get_color() == "синий"

    color.set_color("красный")
    assert color.get_color() == "красный"

@pytest.mark.parametrize("color,expected_error", [
    ("", "Поле цвета не может быть пустым"),
    (None, "Поле цвета не может быть пустым"),
])
def test_color_validation(self, color, expected_error):
    color = FigureColor()
    with pytest.raises(ValueError, match=expected_error):
        color.set_color(color)

```

Скриншот работы программы

```

(venv) uawei_d@DESKTOP-2IUV5GA:~/Yakubova-PCPL-2025$ pytest test.py
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/uawei_d/Yakubova-PCPL-2025
collected 14 items

test.py ..... [100%]

===== 14 passed in 0.03s =====
(venv) uawei_d@DESKTOP-2IUV5GA:~/Yakubova-PCPL-2025$

```